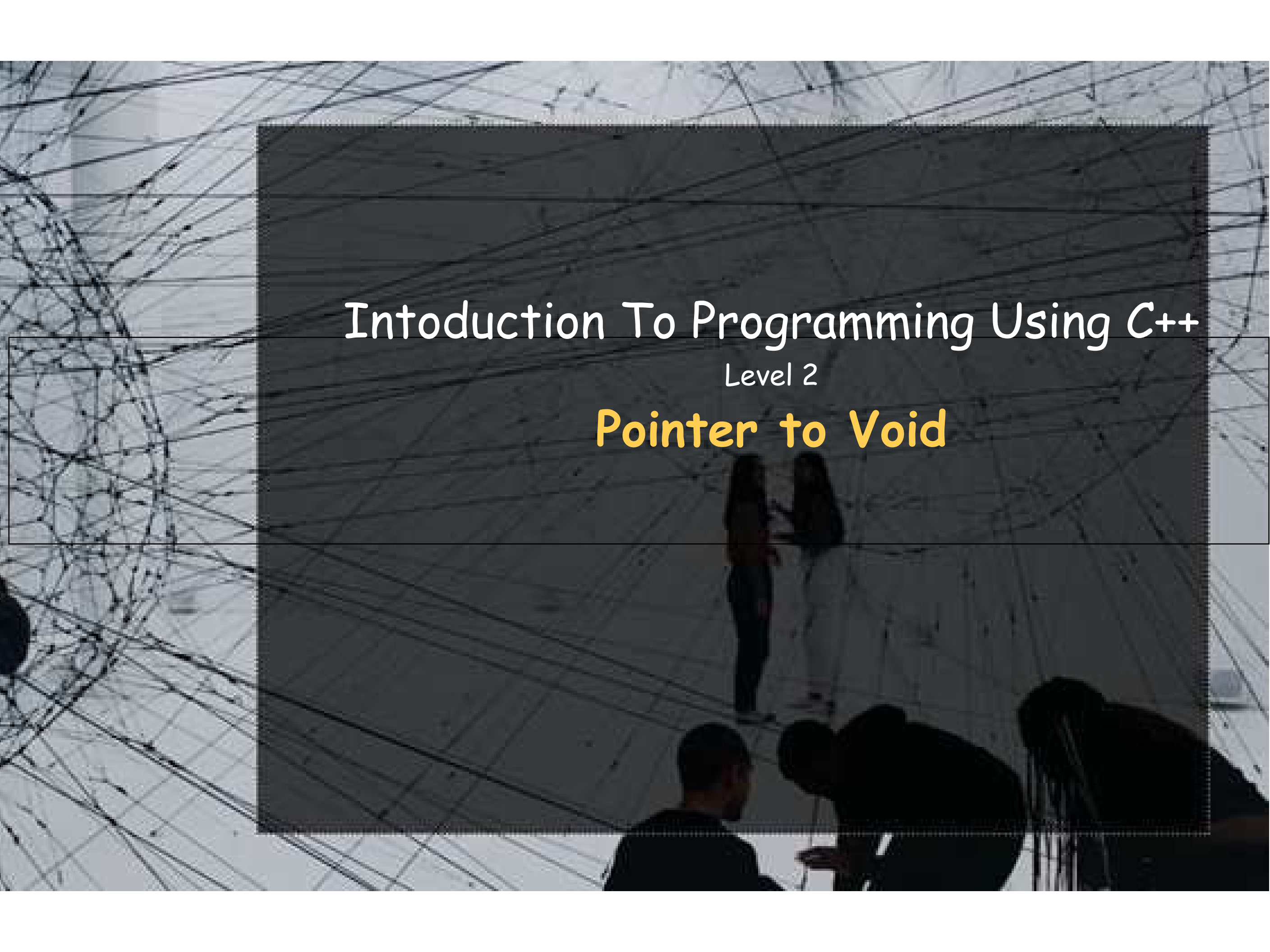


Intoduction To Programming Using C++

Level 2

Pointer to Void



Introduction To Programming Using C++

Level 2

Pointer to Void

Definition

Void Pointer

-  A Void Pointer (``void*``) is a special type of pointer that can store the address of a variable of almost any data type, such as ``int``, ``float``, or ``struct``.
-  However, a ``void*`` does not know the data type of the object it points to.

Explanation

General Pointer

-  A normal pointer has a specific data type.

-  For example:

 - `int* p;`

-  This pointer is designed to point to an `int`.

Void Pointer

-  A `void*` can point to objects of different types.

-  This is useful when we need a general-purpose pointer.

-  However, because the data type is unknown, we cannot directly dereference a `void*`.

-  Before accessing the value, we need to cast the pointer to the correct pointer type.

Procedures

1. Create variables with different data types.
2. Create a ``void*`` pointer.
3. Store the address of an ``int`` variable in the ``void*``.
4. Print the address directly.
5. Cast the ``void*`` to ``int*``.
6. Dereference it to access the ``int`` value.
7. Store the address of a ``float`` variable in the same ``void*``.
8. Cast the ``void*`` to ``float*``.
9. Dereference it to access the ``float`` value.

💡 Example

◆ Void Pointer with `int` and `float`

```
int Number = 10;
float fNumber = 10.53;
void* pointer;
pointer = &Number;
cout << "Number address: " << pointer << endl;
cout << "Number value: " << *(static_cast<int*>(pointer)) << endl;
pointer = &fNumber;
cout << "fNumber address: " << pointer << endl;
cout << "fNumber value: " << *(static_cast<float*>(pointer)) << endl;
```

➡ Result

```
Number address: 0x...
Number value: 10
fNumber address: 0x...
fNumber value: 10.53
```

Note

◆ A ``void*`` cannot be directly dereferenced because the compiler does not know the pointed-to type.

◆ Pointer arithmetic cannot normally be performed directly on a ``void*`` in standard C++:

```
++pointer; // ✗
```

◆ The reason is that the compiler does not know the size of the object to which the ``void*`` points.

◆ After converting it to the correct pointer type, pointer arithmetic can be performed according to that type.

⊘ Warning

- ◆ The cast must match the actual type of the object.

```
int Number = 10;
```

```
void* pointer = &Number;
```

```
cout << *(static_cast<int*>(pointer)) << endl; // ✓ Correct
```

```
cout << *(static_cast<float*>(pointer)) << endl; // ✗ Wrong type
```

- ◆ Casting a `void\*` to the wrong pointer type and then dereferencing it can lead to undefined behavior.



Comparison

▨ C-Style Cast

- ✓ Example: `(int*)pointer`
- ✓ Older C/C++ casting style.
- ✓ Can perform several different kinds of conversions.
- ✓ Makes it less clear what kind of conversion is intended.
- ✓ Generally avoided in modern C++ when a specific cast is available.

▤ `static_cast`

- ✓ Example: `static_cast<int*>(pointer)`
- ✓ Explicitly shows the intended conversion.
- ✓ The compiler checks whether the conversion is permitted by `static_cast`.
- ✓ Preferred over C-style casting when `static_cast` is appropriate.
- ✓ It is not automatically "safe" if the pointer is cast to the wrong type.

✦ Summary

- ✓ A ``void*`` can store the address of objects of different types.
- ✓ A ``void*`` does not know the type of the object it points to.
- ✓ A ``void*`` cannot be directly dereferenced.
- ✓ Cast the ``void*`` to the correct pointer type before dereferencing.
- ✓ ``static_cast`` is preferred over C-style casts when it is appropriate.
- ✓ Always make sure the cast matches the actual type of the object.

Thank you 😊